

Mundo de Wumpus com SARSA

Aula 18 — Aprendizado por Reforço

Breno Porto Pinheiro do Prado RA 185196

Guilherme Bento Ramos RA 185226

1 O Exercício

O objetivo é aplicar o algoritmo **SARSA** ao Mundo de Wumpus clássico, partindo da implementação anterior em Rust (agente BFS baseado em modelo). O trabalho cobre três fases: *(i)* treinar o agente num mapa fixo; *(ii)* testar o modelo treinado num mapa diferente sem re-treinamento (zero-shot); *(iii)* adaptar o modelo ao novo mapa via fine-tuning.

1.1 Ambiente

Grade 4×4 : P=abismo, W=Wumpus, G=ouro. O agente começa em (0,0) voltado para a direita e dispõe de uma flecha. Ele percebe *brisa* (pit adjacente), *cheiro* (Wumpus adjacente) e *brilho* (ouro na casa atual), mas não vê o mapa diretamente. A missão é pegar o ouro e retornar à entrada.

Mapas Os dois mapas foram projetados para testar um aspecto específico de generalização: no mapa de treino o ouro está no extremo **direito** da linha base; no de teste está no extremo **superior esquerdo**. O agente treinado a se mover para a direita tem que reaprender a direção de exploração — um teste de transferência negativa deliberado.

Mapa de treino — ouro em (0,3)

.	.	.	G
.	.	P	.
W	.	.	.
.	.	.	P

(a) Mapa de treino — ouro em (0,3)

Mapa de teste — ouro em (3,0)

.	.	.	.
.	P	.	.
.	.	W	.
G	.	.	.

(b) Mapa de teste — ouro em (3,0)

Figura 1: A=início, G=ouro, P=abismo, W=Wumpus. Linha 0 é a base.

2 SARSA e Configuração

2.1 Algoritmo

SARSA é controle TD on-policy: a atualização usa a ação que *será de fato tomada* no próximo passo (a'), não o máximo sobre todas as ações como no Q-Learning.

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)] \quad (1)$$

Isso torna o agente mais conservador: aprende a considerar o risco de exploração (ϵ -greedy) como parte da política, evitando beiras de abismos durante o treino.

2.2 Representação de estado

$$s = (r, c, d, g, f, w, \textit{brisa}, \textit{cheiro}, \textit{brilho}) \in \mathbb{Z}_4^2 \times \mathbb{Z}_4 \times \{0, 1\}^6 \quad (2)$$

O espaço tem $4 \times 4 \times 4 \times 2^6 = 4096$ estados e $4096 \times 6 = 24\,576$ pares (estado, ação). A tabela Q é um dicionário Python com valor padrão 0; somente os estados visitados ocupam memória.

2.3 Recompensas

Evento	Recompensa total
Passo normal (girar)	−1
Bater na parede	−2
Morte (abismo/Wumpus)	−1001
Flecha acerta Wumpus	+49
Pegar ouro	+99
Sair com ouro	+999
Sair sem ouro	−51
Subir fora da entrada	−11

2.4 Hiperparâmetros

Parâmetro	Valor	Nota
α	0,10	Aprendizado estável sem oscilar
γ	0,95	Valoriza recompensas futuras
$\epsilon_0 / \epsilon_{\min}$	1,0 / 0,02	Exploração total $\rightarrow$ residual
Decaimento de ϵ	0,9995	Chega ao mínimo em $\approx 8\,000$ ep.
Episódios (treino)	10 000	
Episódios (fine-tuning)	5 000	Mapa novo é mais difícil
ϵ no fine-tuning	0,50	Exploração alta para sair do hábito anterior

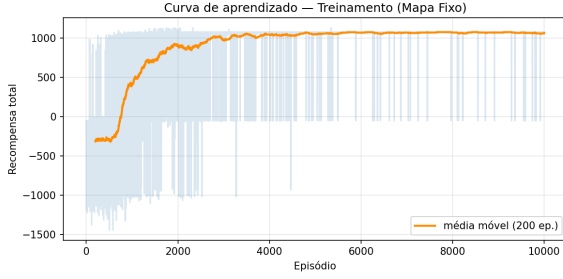
3 Resultados

3.1 Treinamento — mapa fixo

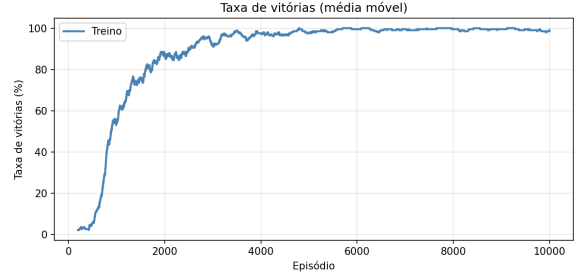
Tabela 1: Avaliação no mapa de treino (200 episódios, $\epsilon = 0$).

Vitórias	200/200 (100%)
Mortes	0/200 (0%)
Recompensa média	1079
Passos médios	11

O agente converge para uma política perfeita. A solução encontrada (11 passos): avança pela linha 0 até o ouro em (0, 3) atravessando (0, 2) com brisa, pega o ouro, gira e retorna. Um tiro é disparado desnecessariamente em (0, 1) — comportamento subótimo herdado de estados em que matar o Wumpus era valioso e que compartilham a mesma entrada na tabela Q.



(a) Recompensa por episódio



(b) Taxa de vitórias

Figura 2: Curvas de aprendizado no mapa de treino (10 000 episódios).

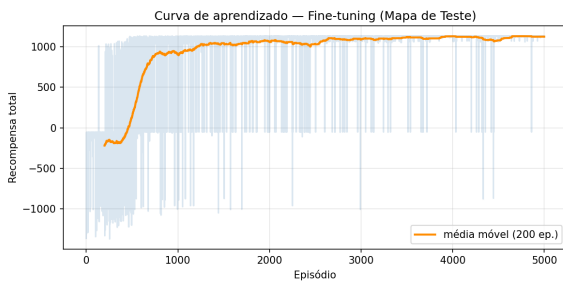
3.2 Transferência zero-shot — mapa de teste

Tabela 2: Avaliação no mapa de teste sem re-treinamento.

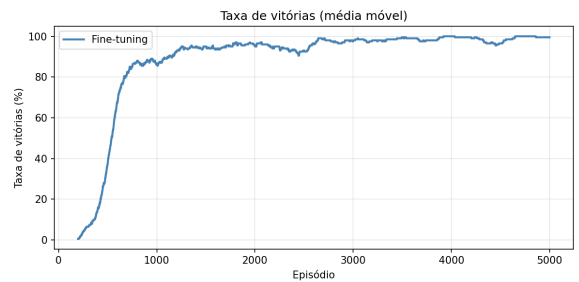
Vitórias	0/200 (0%)
Mortes	181/200 (90,5%)
Timeouts	19/200 (9,5%)
Recompensa média	-1086
Passos médios	62

Zero vitórias. O agente vai direto para a direita (como aprendeu no treino), mas no novo mapa o ouro está em (3,0) — exige ir para **cima**, não para a direita. Após entrar em (0,1) e sentir brisa, fica preso tentando ações do repertório treinado (**climb** fora da entrada, **grab** sem ouro, **shoot** sem alvo) até eventualmente cair no abismo em (1,1). Este é um caso claro de **transferência negativa**: o conhecimento adquirido não apenas não ajuda — ele atrapalha ativamente.

3.3 Fine-tuning — mapa de teste



(a) Recompensa por episódio



(b) Taxa de vitórias

Figura 3: Fine-tuning no mapa de teste (5 000 episódios, $\epsilon_0 = 0.5$).

Com 5 000 episódios adicionais e ϵ reiniciado em 0,5, o agente atinge 100% de vitórias. A solução aprendida (16 passos) mata o Wumpus em (2,2) com tiro pelo corredor da coluna 2, navega até (3,0) ao longo da linha 3 e retorna pela coluna 0. Essa rota é mais longa que o ótimo teórico (11 passos pela coluna 0), mas evita a brisa sentida em (1,0) — o agente preferiu o caminho que parecia mais seguro durante a exploração.

Tabela 3: Avaliação no mapa de teste após fine-tuning.

	Vitórias	200/200 (100%)		
	Mortes	0/200 (0%)		
	Recompensa média	1134		
	Passos médios	16		

Fase	Vitórias	Mortes	Recomp. média	Passos médios
Treino pós-treinamento	100%	0%	1079	11
Teste zero-shot	0%	90,5%	-1086	62
Teste pós fine-tuning	100%	0%	1134	16

4 Análise

4.1 Transferência negativa

A queda de 100% para 0% entre treino e zero-shot não é falha do algoritmo: é consequência de uma mudança *estrutural* no layout do mapa. O estado $(0, 0, \rightarrow)$, sem percepções) tem Q-values altos para **forward** ($\rightarrow$ direita) porque essa ação levava ao ouro no treino. No mapa de teste, o mesmo estado existe mas **forward** leva para longe do ouro.

Os únicos comportamentos que genuinamente generalizam são os **perceptuais**: **grab** quando *brilho* = 1 e **climb** quando em $(0, 0)$ com ouro. Esses nunca são ativados no zero-shot porque o agente nunca chega ao ouro.

4.2 SARSA vs. Q-Learning neste domínio

A característica on-policy do SARSA é uma vantagem prática aqui: durante o treino, quando a política ϵ -greedy explora perto de abismos, o SARSA penaliza esses estados considerando que a próxima ação pode ser aleatória e perigosa. O resultado é uma política mais cautelosa em relação a brisas e cheiros, o que explica a preferência pelo caminho mais longo (mas percebido como mais seguro) no mapa de teste.

4.3 Custo do fine-tuning

O fine-tuning custou 5000 episódios para um mapa com solução de 16 passos. A dificuldade principal foi desfazer o hábito de ir para a direita: nos primeiros 500 episódios as primeiras vitórias mal aparecem. A partir do episódio 1000 a convergência é rápida, indicando que o conhecimento residual (como retornar a $(0, 0)$ depois de pegar o ouro) foi reutilizado e acelerou o aprendizado da nova rota.

4.4 Limitações

- A tabela Q cresce com $O(N^2)$ em grades maiores; DQN seria necessário.
- O reward shaping (bônus por ouro e kill) é necessário para recompensas densas o suficiente; recompensa esparsa (+1000 somente ao final) demandaria exploração muito mais longa.
- Treinar em múltiplos mapas aleatórios (domain randomization) eliminaria a dependência posicional e produziria políticas verdadeiramente generalizáveis.

5 Conclusão

O SARSA convergiu para uma política perfeita no mapa de treino (100%, 11 passos) e revelou transferência negativa completa no mapa de teste (0% zero-shot): o hábito posicional aprendido “vá para a direita” dominou completamente a política e impediu qualquer vitória. O fine-tuning com ϵ elevado desfez o hábito em $\approx 1\,000$ episódios e convergiu para 100% em 5 000, encontrando uma solução de 16 passos que prefere matar o Wumpus e desviar de brisas em vez de seguir o caminho mais curto. O exercício ilustra com clareza o problema de transferência em RL tabular quando o layout muda e por que domain randomization é importante em aplicações reais.

Declaração de uso de IA: este trabalho foi desenvolvido com auxílio do assistente Claude (Anthropic). -2 A IA foi utilizada para estruturar e depurar o código Python, redigir e formatar o relatório em L^AT_EX e discutir a análise dos resultados. Todos os experimentos foram executados localmente e os resultados numéricos reportados são reais, gerados pela execução do arquivo <code>guilherme_ramos_aula18Pratico.py</code>.
